

**Computer Sciences Department
San Jose State University
CS 147 – Introduction to Computer Architecture
Project Specifications**

1 Microarchitecture Specification

In this document, we describe the microarchitecture, including register file specifications, memory system organization, etc. you will use for your CS 147 project. The WISC-SJ26 architecture that you will design for the final project shares many resemblances to the MIPS R2000 described in the text. The major differences are a smaller instruction set and 16-bit words for the WISC-SJ26. Similarities include a load/store architecture and three fixed-length instruction formats.

1.1 Registers

There are eight user registers, R0–R7. Unlike the MIPS R2000, R0 is not always zero. Register R7 is used as the link register for JAL or JALR instructions. The program counter is separate from the user register file. If you choose to implement exceptions for extra credit, a special register named EPC is used to save the current PC upon an exception or interrupt invocation.

1.2 Memory System

The WISC-SJ26 is a Harvard architecture, meaning instructions and data are located in different physical memories. It is byte-addressable, word aligned (where a word is 16 bits long – note that this is different from some of the examples in class), and big-endian. The final version of the WISC-SJ26 will include a multi-cycle memory and one level of cache. However, initial versions of the machine will contain a single cycle memory. See the project deadlines for more details.

The WISC-SJ26 cache replacement policy is deterministic. See the cache module description for an outline of the algorithm you must use.

NOTE: For Phase 1 and Phase 2, you will work with a simplified memory model which supports unaligned accesses.

1.3 Pipeline

The final version of the WISC-SJ26 contains a five-stage pipeline identical to the MIPS R2000. The stages are:

1. Instruction Fetch (IF)
2. Instruction Decode / Register Fetch (ID)
3. Execute / Address Calculation (EX)
4. Memory Access (MEM)
5. Write Back (WB)

See Figure 4.33 on page 299, Figure 4.35 on page 301, or Figure 4.36 on page 303 of the text for good starting points.

1.4 Optimizations

Your goal in optimizations is to reduce the CPI of the processor or the total cycles taken to execute a program. While the primary concern of the WISC-SJ26 is correct functionality, the architecture must still have a reasonable clock period. Therefore, you may not have more than one of the following in series during any stage:

- register file
- memory or cache
- 16-bit full adder
- barrel shifter

You may implement any type of optimization to reduce the CPI (as long as it's a valid optimization). The required optimizations are:

- Register file bypassing
- There are two register forwarding paths in the WISC-SJ26:
 - Forwarding from beginning of the MEM stage to beginning of EX stage (EX $\rightarrow$ EX forwarding)
 - Forwarding from beginning of the WB stage to the beginning of the EX stage (MEM $\rightarrow$ EX forwarding)
- All branches should be predicted not-taken. This means that the pipeline should continue to execute sequentially until the branch resolves, and then squash instructions after the branch if the branch was actually taken.

1.5 Exceptions (extra credit)

NOTE: Exception-handling extra credit will not be graded or evaluated until Phase 3 of the project. However, students are encouraged to try to work exception support into their design earlier.

Exception handling is extra credit. If you choose not to implement exception handling, an illegal instruction should be treated as a NOP. IllegalOp is the only defined exception in the WISC-SJ26 architecture. It is invoked when the opcode of the currently executing instruction is not a recognized member of the ISA. Upon finding an illegal opcode, the computer shall save the current PC into the reserved register EPC and then load address 0x02, which is the location of the IllegalOp exception handler. Note that if you choose to implement exceptions, address 0x00 must be a jump to the start of the main program.

The exception handler itself need not be complex. At a minimum it should load the value 0xBADD into R7 and then use/call the RTI instruction to return to the address specified by the EPC.

2 ISA Specification

2.1 Instruction Summary

KEY: sss = rs, ddd = rd, ttt = rt, iii* = immediate.

Encoding	Format	Syntax	Semantics
00000	xxxxxxxxxxx	HALT	Cease instruction issue, dump memory state to file
00001	xxxxxxxxxxx	NOP	None
01000	sss ddd iiiii	ADDI Rd, Rs, immediate	Rd <- Rs + I(sign ext.)
01001	sss ddd iiiii	SUBI Rd, Rs, immediate	Rd <- I(sign ext.) - Rs
01010	sss ddd iiiii	XORI Rd, Rs, immediate	Rd <- Rs XOR I(zero ext.)
01011	sss ddd iiiii	ANDNI Rd, Rs, immediate	Rd <- Rs AND ~I(zero ext.)
10100	sss ddd iiiii	ROLI Rd, Rs, immediate	Rd <- Rs «(rotate) I(lowest 4 bits)
10101	sss ddd iiiii	SLLI Rd, Rs, immediate	Rd <- Rs « I(lowest 4 bits)
10110	sss ddd iiiii	RORI Rd, Rs, immediate	Rd <- Rs »(rotate) I(lowest 4 bits)
10111	sss ddd iiiii	SRLI Rd, Rs, immediate	Rd <- Rs » I(lowest 4 bits)
10000	sss ddd iiiii	ST Rd, Rs, immediate	Mem[Rs + I(sign ext.)] <- Rd
10001	sss ddd iiiii	LD Rd, Rs, immediate	Rd <- Mem[Rs + I(sign ext.)]
10011	sss ddd iiiii	STU Rd, Rs, immediate	Mem[Rs + I(sign ext.)] <- Rd; Rs <- Rs + I(sign ext.)
11001	sss xxx ddd xx	BTR Rd, Rs	Rd[bit i] <- Rs[bit 15-i] for i=0..15
11011	sss ttt ddd 00	ADD Rd, Rs, Rt	Rd <- Rs + Rt
11011	sss ttt ddd 01	SUB Rd, Rs, Rt	Rd <- Rt - Rs
11011	sss ttt ddd 10	XOR Rd, Rs, Rt	Rd <- Rs XOR Rt
11011	sss ttt ddd 11	ANDN Rd, Rs, Rt	Rd <- Rs AND ~Rt
11010	sss ttt ddd 00	ROL Rd, Rs, Rt	Rd <- Rs « (rotate) Rt (lowest 4 bits)
11010	sss ttt ddd 01	SLL Rd, Rs, Rt	Rd <- Rs « Rt (lowest 4 bits)
11010	sss ttt ddd 10	ROR Rd, Rs, Rt	Rd <- Rs » (rotate) Rt (lowest 4 bits)
11010	sss ttt ddd 11	SRL Rd, Rs, Rt	Rd <- Rs » Rt (lowest 4 bits)
11100	sss ttt ddd xx	SEQ Rd, Rs, Rt	if (Rs == Rt) then Rd <- 1 else Rd <- 0
11101	sss ttt ddd xx	SLT Rd, Rs, Rt	if (Rs < Rt) then Rd <- 1 else Rd <- 0

Encoding	Format	Syntax	Semantics
11110	sss ttt ddd xx	SLE Rd, Rs, Rt	if (Rs ≤ Rt) then Rd ← 1 else Rd ← 0
11111	sss ttt ddd xx	SCO Rd, Rs, Rt	if (Rs + Rt) generates carry out, then Rd ← 1 else Rd ← 0
01100	sss iiiiiiiii	BEQZ Rs, signed immediate	if (Rs == 0) then PC ← PC + 2 + I(sign ext.)
01101	sss iiiiiiiii	BNEZ Rs, signed immediate	if (Rs != 0) then PC ← PC + 2 + I(sign ext.)
01110	sss iiiiiiiii	BLTZ Rs, signed immediate	if (Rs < 0) then PC ← PC + 2 + I(sign ext.)
01111	sss iiiiiiiii	BGEZ Rs, signed immediate	if (Rs ≥ 0) then PC ← PC + 2 + I(sign ext.)
11000	sss iiiiiiiii	LBI Rs, signed immediate	Rs ← I(sign ext.)
10010	sss iiiiiiiii	SLBI Rs, unsigned im- mediate	Rs ← (Rs « 8) I(zero ext.)
00100	ddddddddddd	J displacement	PC ← PC + 2 + D(sign ext.)
00101	sss iiiiiiiii	JR Rs, signed immediate	PC ← Rs + I(sign ext.)
00110	ddddddddddd	JAL displacement	R7 ← PC + 2; PC ← PC + 2 + D(sign ext.)
00111	sss iiiiiiiii	JALR Rs, signed immediate	R7 ← PC + 2; PC ← Rs + I(sign ext.)
00010	siic Rs	SIIC Rs	produce IllegalOp exception. Must provide one source register.
00011	xxxxxxxxxxx	NOP / RTI	PC ← EPC

2.2 Formats

WISC-SJ26 supports instructions in four different formats: J-format, 2 I-formats, and the R-format. These are described below.

2.2.1 J-format

The J-format is used for jump instructions that need a large displacement.

5 bits	11 bits
Op Code	Displacement

Jump Instructions. The Jump instruction loads the PC with the value found by adding the PC of the next instruction (PC+2, not PC+4 as in MIPS) to the sign-extended displacement. The Jump-And-Link instruction loads the PC with the same value and also saves the address of the next sequential instruction (i.e., PC+2) in the link register R7.

The syntax of the jump instructions is:

- J displacement
- JAL displacement

2.2.2 I-format

I-format instructions use either a destination register, a source register, and a 5-bit immediate value; or a destination register and an 8-bit immediate value. The two types of I-format instructions are described below.

I-format 1

5 bits	3 bits	3 bits	5 bits
Op Code	Rs	Rd	Immediate

The I-format 1 instructions include XOR-Immediate, ANDN-Immediate, Add-Immediate, Subtract-Immediate, Rotate-Left-Immediate, Shift-Left-Logical-Immediate, Rotate-Right-Immediate, Shift-Right-Logical-Immediate, Load, Store, and Store with Update.

The syntax of the I-format 1 instructions is:

- ADDI Rd, Rs, immediate
- SUBI Rd, Rs, immediate
- XORI Rd, Rs, immediate
- ANDNI Rd, Rs, immediate
- ROLI Rd, Rs, immediate
- SLLI Rd, Rs, immediate
- RORI Rd, Rs, immediate
- SRLI Rd, Rs, immediate
- ST Rd, Rs, immediate
- LD Rd, Rs, immediate
- STU Rd, Rs, immediate

I-format 2

5 bits	3 bits	8 bits
Op Code	Rs	Immediate

The format of these instructions is:

- LBI Rs, signed immediate
- SLBI Rs, unsigned immediate
- JR Rs, immediate
- JALR Rs, immediate
- BEQZ Rs, signed immediate
- BNEZ Rs, signed immediate
- BLTZ Rs, signed immediate
- BGEZ Rs, signed immediate

2.2.3 R-format

5 bits	3 bits	3 bits	3 bits	2 bits
Op Code	Rs	Rt	Rd	Op Code Extension

The syntax of the R-format ALU and shift instructions is:

- ADD Rd, Rs, Rt
- SUB Rd, Rs, Rt
- XOR Rd, Rs, Rt
- ANDN Rd, Rs, Rt
- ROL Rd, Rs, Rt
- SLL Rd, Rs, Rt
- ROR Rd, Rs, Rt
- SRL Rd, Rs, Rt
- SEQ Rd, Rs, Rt
- SLT Rd, Rs, Rt
- SLE Rd, Rs, Rt
- SCO Rd, Rs, Rt
- BTR Rd, Rs

2.3 Special Instructions

Special instructions use the R-format. The syntax of these instructions is:

- HALT
- NOP

The SIIC and RTI instructions are extra credit and can be deferred for later. They will not be tested until the final demo.

3 Cache Module Specification

3.1 Cache Interface

This figure shows the external interface to the module. Each signal is described in the table below.

Signal	In/Out	Width	Description
enable	In	1	Enable cache. Active high. If low, "write" and "comp" have no effect, and all outputs are zero.
index	In	8	The address bits used to index into the cache memory.
offset	In	3	offset[2:1] selects which word to access in the cache line. The least significant bit should be 0 for word alignment. If the least significant bit is 1, it is an error condition.
comp	In	1	Compare. When comp=1, the cache will compare tag_in to the tag of the selected line and indicate if a hit has occurred; the data portion of the cache is read or written but writes are suppressed if there is a miss. When comp=0, no compare is done and the Tag and Data portions of the cache will both be read or written.
write	In	1	Write signal. If high at the rising edge of the clock, a write is performed to the data selected by "index" and "offset", and (if comp=0) to the tag selected by "index".
tag_in	In	5	When comp=1, this field is compared against stored tags to see if a hit occurred; when comp=0 and write=1 this field is written into the tag portion of the array.
data_in	In	16	On a write, the data that is to be written to the location specified by the index and offset inputs.
valid_in	In	1	On a write when comp=0, the data that is to be written to valid bit at the location specified by the index input.
clk	In	1	Clock signal; rising edge active.
rst	In	1	Reset signal. When rst=1 on the rising edge of the clock, all lines are marked invalid. (The rest of the cache state is not initialized and may contain X's.)
createdump	In	1	Write contents of entire cache to memory file. Active on rising edge.
hit	Out	1	Goes high during a compare if the tag at the location specified by the index lines matches the tag_in lines.

Signal	In/Out	Width	Description
dirty	Out	1	When this bit is read, it indicates whether this cache line has been written to. It is valid on a read cycle, and also on a compare-write cycle when hit is false. On a write with <code>comp=1</code> , the cache sets the dirty bit to 1. On a write with <code>comp=0</code> , the dirty bit is reset to 0.
tag_out	Out	5	When <code>write=0</code> , the tag selected by index appears on this output. (This value is needed during a write-back.)
data_out	Out	16	When <code>write=0</code> , the data selected by index and offset appears on this output.
valid	Out	1	During a read, this output indicates the state of the valid bit in the selected cache line.
err	Out	1	Error output (used for invalid offset[0]).

3.2 Instantiating Cache Modules

When instantiating the module, there is a parameter which should be set for each instance. When you dump the contents of the cache to a set of files (e.g. for debugging), this parameter allows each instance to go to a unique set of filenames.

Parameter Value	File Names
0	Icache_0_data_0, Icache_0_data_1, Icache_0_tags, ...
1	Dcache_0_data_0, Dcache_0_data_1, Dcache_0_tags, ...
2	Icache_1_data_0, Icache_1_data_1, Icache_1_tags, ...
3	Dcache_1_data_0, Dcache_1_data_1, Dcache_1_tags, ...

3.3 Organization of the Cache

The cache contains 256 lines. Each line contains one valid bit, one dirty bit, a 5-bit tag, and four 16-bit words.

3.4 Cache Operation and Semantics (Direct-mapped Cache)

Although there are a lot of signals for the cache, its operation is pretty simple. When `enable` is high, the two main control lines are `comp` and `write`. Here are the four cases:

1. Compare Read (`comp=1`, `write=0`)
2. Compare Write (`comp=1`, `write=1`)

3. Access Read (`comp=0, write=0`)
4. Access Write (`comp=0, write=1`)

3.5 Building a Two-way Set-associative Cache

After you have a working design using a direct-mapped cache, you will add a second cache module to make your design two-way set-associative.

3.6 Pseudo-random Replacement for Two-way Set-associative Cache

In order to make the designs more deterministic and easier to grade, all set-associative caches must implement the following replacement algorithm:

1. Have a flip-flop called `victimway` which is initialized to zero.
2. On each read or write of the cache, invert the state of `victimway`.
3. When installing a line after a cache miss, install in an invalid block if possible. If both ways are invalid, install in way zero.
4. If both ways are valid, and a block must be victimized, use `victimway` (after already being inverted for this access) to indicate which way to use.
5. For the D cache, do not invert `victimway` for instructions that do not read or write cache, or for invalid instructions, or for instructions that are squashed due to branch misprediction.
6. For the I cache, invert `victimway` for each instruction fetched.

Note

This document is subject to change as new phases of the project become active. If an updated version is available, you can acquire it via `./run update`.